

RAILSYNC — High-Precision Operational Intelligence

Dynamic Train Arrival Forecasting, Multi-Factor Delay Decomposition & Operational Resilience System

Author / Project: RailSync Core Team	Version: 2.0 Production Ready	Target Domain: Indian Railways (IR)	Date: 15 September 2026
--------------------------------------	-------------------------------	-------------------------------------	-------------------------

1. Executive Summary & Problem Statement (Kyun Banaya?)

The Reality of Current Train Tracking in India:
Existing apps like *Where Is My Train* and *NTES (National Train Enquiry System)* only display **past historical delays** based on manual station punches. For instance, if Train 12301 is 45 minutes late leaving Kanpur, these apps simply shift all downstream station arrival times by exactly +45 minutes.

Why this fails in real railway operations:

- **Slack Recovery (Time Deletion):** High-priority trains with WAP-7 locomotives run at 130 km/h on clear track, often recovering 15 to 30 minutes against scheduled timetable slack.
- **Fog & Visibility:** In North India (Gangetic Belt), dense fog drops visibility below 50m, clamping maximum speed to 60 km/h (+17.5m delay/100km).
- **Terminal Platform Contention:** Even if a train is running on time, if its designated platform at New Delhi (NDLS) is occupied, it gets held at the outer-home signal for 15-20 minutes.

The RailSync Solution: RailSync replaces static delay projection with a **physics-driven, multi-factor dynamic prediction engine** combining real-time ISRO/GPS telemetry, live aviation METAR weather feeds, convoy headway gap monitoring, and yard conflict resolution.

2. System Architecture & Dual-Engine Pipeline

RailSync operates on a **Dual-Engine Architecture** to guarantee 100% operational availability whether online with live GPS satellite feeds or offline running high-fidelity railway physics simulation:

Subsystem	Mechanism / Protocol	Operational Role & Output
Option B: Live API	REST HTTP Client indianrailapi.com v2	Fetches real-time GPS & station punch data for any 5-digit train in India . Dead-reckoning algorithm compensates for stale station punches.
Kinematic Physics Engine	WAP-7 Tractive Curves Sectional MPS (130/110)	Simulates high-precision train movement using locomotive tractive effort, corridor elevation gradients, track curvature, and timetable buffers when offline.
WeatherEngine	Aviation METAR Feeds VIDP, VILK, VECC	Continuously monitors visibility, precipitation rates, and extreme heat. Dynamically injects speed clamps (e.g. MPS 60 in fog) into ETA forecasts.
NetworkTracker	Convoy Headway Analysis Great-Circle Haversine	Tracks train ahead (e.g. Train 12876 Neelachal Exp). Computes headway gap (km) and enforces automatic 4-aspect signal caution restrictions.
Incident Detector	Rolling Window Anomaly Crowdsourced Reports	Detects unscheduled stops (>300s below 5 km/h) or rapid decelerations (>50 km/h drop in 60s). Aggregates verified passenger incident reports.
Station Operations	Gantt Occupancy Matrix Two-Way REST Resolution	Identifies outer-home holding delays due to platform congestion. Allows station controllers to reassign platforms and instantly delete outer holding delay.

3. Deep Dive: The 5 Core Technical Innovations

Pillar 1: Multi-Factor Waterfall Delay Decomposition

Instead of outputting an opaque delay number, RailSync mathematically decomposes total arrival delay into **5 distinct operational buckets**:

Net Arrival Delay Formula:

$$\text{Net Delay} = T_{\text{weather}} + (T_{\text{signals}} + T_{\text{tsr}}) + T_{\text{throat}} + T_{\text{platform}} - T_{\text{recovery}}$$

- T_{weather} : Atmospheric speed restrictions (Fog, Heavy Monsoon Rain, Extreme Heat).
- $T_{\text{signals}} + T_{\text{tsr}}$: Headway braking behind leading train + active divisional caution orders (track work).
- T_{throat} : Diamond crossover friction entering major station junctions (25 km/h restriction).
- T_{platform} : Outer-home holding due to platform contention.
- T_{recovery} (**Time Deletion**): Recovered minutes running at MPS (130 km/h) against timetable slack buffer.

Pillar 2: Dynamic Time Deletion (Slack Recovery) vs Time Delay Injection

Every train schedule in Indian Railways contains *slack buffer time* (5–12% padded runtime). RailSync actively calculates section-by-section whether a train is losing time or regaining it:

- **Time Delay Injection (+mins)**: Injected when TSR caution orders or fog slow the train down.
- **Time Deletion (-mins)**: Subtracted when a high-priority train (Rajdhani/Superfast) sustains MPS 130 km/h over long uninterrupted blocks (e.g. Kanpur to New Delhi).

Both the web dashboard and CLI display explicit tags: Time Del (-3.4m) or Delay Inj (+6.0m).

Pillar 3: Real-Time Atmospheric Radar & Weather Clamping

The WeatherEngine queries live METAR aviation airport stations along the corridor. It automatically applies operational safety rules calibrated from IR Operating Manuals:

Condition	Threshold	Delay Penalty / 100km	Locomotive Operational Impact
Severe Fog	Visibility < 50 m	+17.5 min	MPS clamped to 60 km/h; detonator fog signals deployed.
Moderate Fog	Visibility 50–200 m	+4.0 min	MPS clamped to 90 km/h; caution approach to home signals.
Severe Rain	Precipitation > 50 mm/hr	+12.5 min	Flash flood caution order; wheel-rail adhesion reduction.
Extreme Heat	Temperature > 50°C	+5.0 min	CWR track buckling hazard; daytime thermal speed restrictions.

Pillar 4: Live Indian Rail API Integration (Option B)

RailSync connects directly to `http://indianrailapi.com/api/v2/livetrainstatus/...`

- Dynamically supports **ANY 5-digit Indian Railways train number** across all 18 railway zones.
- **Dead-Reckoning Algorithm**: If the last station punch was 15 minutes ago, RailSync calculates how many kilometers the train has traveled based on its sectional speed profile to display accurate real-time coordinates.

Pillar 5: Platform Contention Resolution & Anomaly AI

- **Outer-Home Holding Elimination**: If Train 12301 and Train 12876 are both routed to Platform 12 at New Delhi, the Station Operations UI flags a platform conflict (+10 min hold). With 1-click conflict resolution, reassigning to Platform 16 instantly clears the holding penalty across all downstream ETAs.
- **Incident Anomaly Detector**: Analyzes rolling 15-minute telemetry streams to identify unauthorized chain pulling, cattle run-over, or track obstructions with anti-spam passenger crowdsourcing.

4. Technology Stack & Automated Verification Suite

Stack Layer	Technologies & Libraries	Codebase Location	Test Coverage
Backend API	Python 3.10, FastAPI, Uvicorn, Pydantic v2	api/main.py, api/schemas.py	79% (8 endpoints tested)
Prediction Engine	Kinematic Physics, NumPy, LightGBM	engine/predictor.py, engine/replay.py	86% (14 test scenarios)
Weather Integration	Aviation METAR feeds, Climatological model	engine/weather_engine.py	77% (8 unit tests)
Network & Congestion	Haversine Great-Circle, Convoy Tracker	engine/network_tracker.py	84% (5 integration tests)
Incident Detector	Rolling Anomaly Detection, Anti-Spam	engine/incident_detector.py, api/incidents.py	98% (9 edge tests)
Live Indian Rail API	urllib HTTP client, Dead-Reckoning	engine/live_rail_api.py	75% (Live + mock tests)
Frontend Web App	React 19, TypeScript, Vite, Tailwind CSS, Lucide	src/App.tsx, src/services/railSyncApi.ts	0 TypeScript errors
Terminal CLI	Argparse, ANSI Terminal Color Formatter	cli.py	Interactive & arg modes

Automated Test Suite Status (100% Green):

- **70 Passed out of 70 Tests** in 25.71 seconds.
- **Total Codebase Coverage: 84%** across all 1,969 statements.
- Verified zero regressions across: Weather speed clamping, NetworkTracker convoy spacing, Incident anomaly thresholds, Crowdsourcing anti-spam cooldown, Dynamic station waterfall decomposition, and REST endpoint serialization.

5. Team 2-Minute Demo Cheat-Sheet (Presentation Guide)

Use this exact 3-step sequence to present RailSync to professors, evaluators, or team members:

Step 1: Run Any Dynamic Train in CLI (No GUI Needed)

```
# 1. Terminal Demo: Any 5-Digit Indian Railways Train Number python cli.py --train 12951 # (With Live API Key):  
python cli.py --train 12951 --api-key
```

Talking Point: 'RailSync dynamically resolves Train 12951 as Mumbai Rajdhani (MPS 130 km/h, Priority 1), pulls live METAR weather, and breaks down the exact downstream waterfall showing Time Deletion (-m) and Delay Injection (+m).'

Step 2: Demonstrate Automated Verification Rigor

```
# 2. Run Test Suite to Prove System Rigor $env:PYTEST_DISABLE_PLUGIN_AUTOLOAD=1; python -m pytest tests/ -q  
Talking Point: 'Every module is backed by 70 automated tests with 84% code coverage, ensuring zero broken dependencies.'
```

Step 3: Launch Full-Stack Dashboard

```
# Terminal 1: Launch FastAPI Backend uvicorn api.main:app --reload --port 8000 # Terminal 2: Launch React 19 Frontend npm run dev
```

Talking Point: 'The frontend renders real-time convoy headway gaps on the GIS map, telemetry streaming from RTIS GPS, and allows station controllers to perform interactive two-way platform conflict resolution in real time.'